

Media Service (STUDER-MEDIA)

▼ Relevant source files

- Backend/STUDER-MEDIA/Dockerfile
- Backend/STUDER-MEDIA/main.py
- Backend/STUDER-MEDIA/placeholder.txt
- Backend/STUDER-MEDIA/requirements.txt
- Backend/STUDER/src/main/java/facu/studer/DTOs/media/ImageUploadResponseDTO.java

The **STUDER-MEDIA** service is a specialized microservice built with **Python FastAPI** `Backend/STUDER-MEDIA/main.py` 16 dedicated to image processing and storage management. It acts as a middleware between the Spring Boot backend and the MinIO object storage, ensuring that all uploaded images are optimized, resized, and converted into appropriate formats for different UI contexts (e.g., avatars, thumbnails, high-resolution previews).

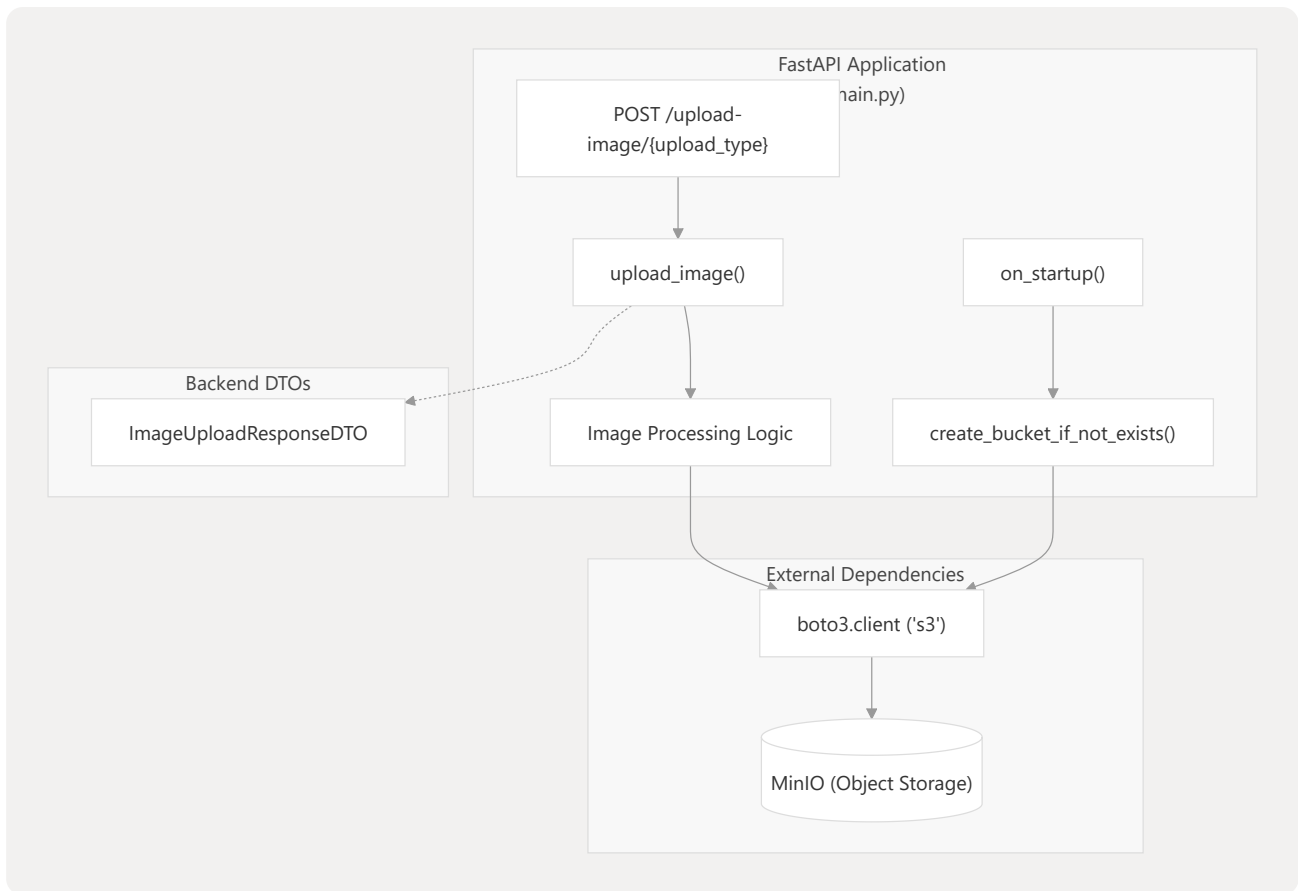
Service Architecture

The service utilizes the **Pillow** (PIL) library for high-performance image manipulation `Backend/STUDER-MEDIA/requirements.txt` 3 and **boto3** for seamless integration with S3-compatible storage `Backend/STUDER-MEDIA/requirements.txt` 5 It is containerized using a lightweight Python slim image `Backend/STUDER-MEDIA/Dockerfile` 1 and exposed via **uvicorn** `Backend/STUDER-MEDIA/Dockerfile` 10

Service Interaction Map

The following diagram illustrates how the **STUDER-MEDIA** service maps internal logic to the broader system infrastructure.

Image Processing & Storage Context



Sources: Backend/STUDER-MEDIA/main.py 93-102 Backend/STUDER-MEDIA/main.py 34-40
Backend/STUDER/src/main/java/facu/studer/DTOS/media/ImageUploadResponseDTO.java 6-9

Image Upload Pipeline

The core functionality is centered around the `upload_image` endpoint

Backend/STUDER-MEDIA/main.py 97-102 Depending on the `upload_type` path parameter, the service triggers specific transformation pipelines:

- **Profile Images:** Generates four variants: the `original` file, a `1080p avatar`, a modern `WebP` version, and a small `150x150 thumbnail` Backend/STUDER-MEDIA/main.py 111-144
- **Message Images:** Generates two variants: the `original` file and a `500x500 thumbnail` Backend/STUDER-MEDIA/main.py 146-162

All pipelines include an automatic **RGBA to RGB conversion** to ensure compatibility with JPEG formatting when transparency is present in the source Backend/STUDER-MEDIA/main.py 119-121

For details on transformation logic and DTO mapping, see [Image Upload Pipeline](#).

MinIO Object Storage Integration

The service integrates with **MinIO** using S3-compatible protocols. Configuration is driven by environment variables such as `MINIO_ENDPOINT` and `BUCKET_NAME`

Backend/STUDER-MEDIA/main.py 18-22

Storage Lifecycle

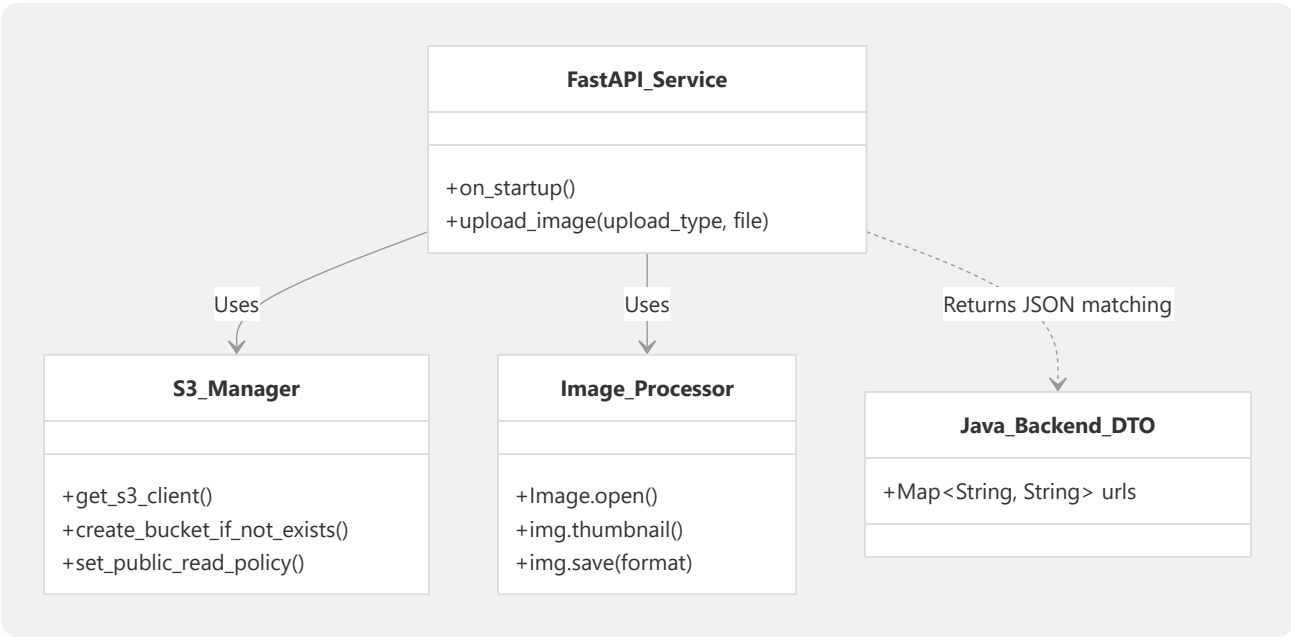
1. **Initialization:** On startup, the service attempts to create the bucket if it does not exist, with a 5-attempt retry logic Backend/STUDER-MEDIA/main.py 60-90
2. **Security:** It automatically applies a **Public Read Policy** (`s3:GetObject`) to the bucket to allow the frontend to fetch images directly via URL Backend/STUDER-MEDIA/main.py 42-53
3. **Persistence:** Images are stored in structured paths (e.g., `profile/avatar/filename.jpg`) and the resulting URLs are returned to the backend Backend/STUDER-MEDIA/main.py 126-127

For details on bucket policies and S3 client configuration, see [MinIO Object Storage Integration](#).

Code Entity Association

This diagram bridges the functional requirements of the media service to the specific code entities responsible for implementation.

Media Service Entity Mapping



Sources: Backend/STUDER-MEDIA/main.py 30-40 Backend/STUDER-MEDIA/main.py 97-102

Backend/STUDER-MEDIA/main.py 117-124

Backend/STUDER/src/main/java/facu/studer/DTOs/media/ImageUploadResponseDTO.java 1-9